

100+ ML Interview Questions

The complete, no-fluff prep guide — every question a Google, Meta, Amazon or product-based interviewer actually asks, answered the way a senior candidate would.

■ Fundamentals	Q1-Q20
■ Supervised Learning	Q21-Q40
■ Unsupervised Learning	Q41-Q55
■ Model Evaluation	Q56-Q70
■ Feature Engineering	Q71-Q82
■ Deep Learning & NLP	Q83-Q95
■ ML System Design	Q96-Q110

Not definitions. Decisions. Every answer here connects the concept to a real trade-off, so you sound like someone who has shipped models — not just read about them.

WHAT'S INSIDE

Table of Contents

01

ML Fundamentals Q1 - Q20

Core concepts every ML practitioner must know

02

Supervised Learning Q21 - Q40

Regression, classification, algorithms, evaluation

03

Unsupervised Learning Q41 - Q55

Clustering, dimensionality reduction, anomaly detection

04

Model Evaluation Q56 - Q70

Metrics, validation, bias-variance, business alignment

05

Feature Engineering Q71 - Q82

The skill that separates good models from bad ones

06

Deep Learning & NLP Q83 - Q95

Neural networks, transformers, practical DL questions

07

ML System Design Q96 - Q110

End-to-end thinking — the senior-level differentiator

How to use this guide: read one section a day, close the PDF, and explain each answer out loud in under 60 seconds. If you can't, you don't know it yet — you're pattern-matching.

SECTION 01 • Q1 – Q20

ML Fundamentals

Get these wrong and the interview ends early. Get these right and you've cleared the first filter most candidates fail.

Q1

What is the difference between supervised, unsupervised, and reinforcement learning?

Type	Has labels?	Goal	Examples
Supervised	Yes	Predict a label from inputs	Classification, regression
Unsupervised	No	Find structure in data	Clustering, dimensionality reduction
Reinforcement	Environment reward	Maximise cumulative reward	Game playing, robotics, recommendation

In an interview: always close with a real example — “supervised is predicting whether a customer will churn, given their behaviour history.”

Q2

What is the bias-variance tradeoff?

Bias is error from wrong assumptions — high bias means the model is too simple (underfitting). Variance is error from sensitivity to the training data — high variance means the model is too complex (overfitting). Reducing one tends to raise the other; the goal is minimising total error, not either term alone.

	High Bias	High Variance
Training error	High	Low
Validation error	High	Much higher than training
Problem	Underfitting	Overfitting
Fix	More complex model, more features	Regularisation, more data, simpler model

Q3

What is overfitting and how do you prevent it?

Overfitting is when a model learns the training data too well — including its noise — and fails to generalise to new data.

Prevention:

- More data — the single best fix
- Cross-validation — confirm performance holds on unseen splits
- Regularisation — L1 (Lasso) and L2 (Ridge) penalise complexity
- Dropout — randomly drop neurons during training (neural nets)
- Early stopping — halt once validation loss stops improving
- Simpler model — fewer parameters, shallower trees
- Feature selection — remove irrelevant or noisy features

Q4

What is the difference between parameters and hyperparameters?

	Parameters	Hyperparameters
What	Learned by the model during training	Set by you before training
Examples	Weights in a neural net, regression coefficients	Learning rate, tree depth, number of clusters
How set	Gradient descent / optimisation	Grid search, random search, Bayesian optimisation

Q5

Explain the curse of dimensionality.

As the number of features grows, the volume of the feature space grows exponentially, so:

- Data becomes sparse — points sit far apart from one another
- Distance metrics lose meaning — everything drifts toward equidistant
- Models need exponentially more data to generalise
- Overfitting risk climbs sharply

Fix: dimensionality reduction (PCA, t-SNE), feature selection, regularisation.

Q6

What is cross-validation and why do we use it?

Cross-validation evaluates generalisation by training and testing on different subsets of the data.

```
K-Fold CV (k=5):  
Fold 1: [Test ] [Train] [Train] [Train] [Train]  
Fold 2: [Train] [Test ] [Train] [Train] [Train]  
Fold 3: [Train] [Train] [Test ] [Train] [Train]  
...repeat for all k folds, then average the results
```

Method	When to use
K-Fold (k=5 or 10)	Standard default
Stratified K-Fold	Class imbalance — preserves class proportions
Time Series Split	Time-ordered data — never leak future into the past
Leave-One-Out	Very small datasets

Q7

What is the train/validation/test split, and why do you need all three?

Set	Used for	Typical size
Training	Fit model parameters	60-70%
Validation	Tune hyperparameters, select model	10-20%
Test	Final, unbiased evaluation — touch once	10-20%

Critical: if you peek at the test set during development it becomes a second validation set, and your final number is optimistically biased. The test set is sacred.

Q8 What is gradient descent?

An optimisation algorithm that minimises a loss function by iteratively nudging parameters in the direction of steepest descent:

$$w = w - \text{learning_rate} \times \partial \text{Loss} / \partial w$$

Variant	Data used per update	Speed	Stability
Batch GD	All data	Slow	Stable
Stochastic GD	One sample	Fast	Noisy
Mini-batch GD	Small batch (32-512)	Balanced	Balanced

Q9 What is the learning rate, and why does it matter?

- Too high — overshoots the minimum; loss oscillates or diverges
- Too low — converges painfully slowly, can stall in a local minimum
- Best practice — start around 1e-3, use a scheduler, or let Adam adapt it automatically

Q10 What is regularisation? Explain L1 vs L2.

Regularisation adds a penalty to the loss function to discourage overly complex models.

	L1 (Lasso)	L2 (Ridge)
Penalty	Sum of absolute weights	Sum of squared weights
Effect on weights	Drives some exactly to zero	Shrinks all weights proportionally
Feature selection	Yes — sparse solution	No — keeps all features
Best when	Many irrelevant features	All features are somewhat useful

ElasticNet: blends L1 and L2 when you want a bit of both.

Q11 What is the difference between classification and regression?

	Classification	Regression
Output	Discrete class label	Continuous numeric value
Examples	Fraud / not fraud, churn / not churn	House price, sales forecast
Loss function	Cross-entropy, log loss	MSE, MAE, RMSE
Evaluation	Accuracy, precision, recall, AUC	RMSE, MAE, R^2

Q12 What is a loss function? Give examples.

A loss function measures how wrong the model's predictions are — training minimises it.

Loss	Used for	Idea
MSE	Regression	Mean of squared errors
MAE	Regression (outlier-robust)	Mean of absolute errors
Binary cross-entropy	Binary classification	Penalises confident wrong answers
Categorical cross-entropy	Multi-class	Same idea across many classes
Hinge loss	SVM	Penalises points inside the margin

Q13 Explain the difference between generative and discriminative models.

	Generative	Discriminative
Models	Joint distribution $P(X, Y)$	Conditional distribution $P(Y X)$
Examples	Naive Bayes, GANs, VAEs	Logistic Regression, SVM, neural nets
Can generate new data?	Yes	No
Usually better at classification?	Less so	Yes

Q14 What is feature scaling, and when is it necessary?

Method	Formula	Use when
Standardisation (Z-score)	$(x - \text{mean}) / \text{std}$	Roughly normal data, SVM, linear models
Min-Max normalisation	$(x - \text{min}) / (\text{max} - \text{min})$	Bounded range needed, neural nets

Method	Formula	Use when
Robust scaling	$(x - \text{median}) / \text{IQR}$	Outliers present
Required for: gradient descent, SVM, KNN, PCA, neural nets. Not required for: tree-based models (Random Forest, XGBoost, Decision Tree).		

Q15 What is data leakage, and why is it dangerous?

Data leakage is information from outside the training set contaminating the model, producing artificially high scores that collapse in production.

Common forms:

- Scaling the whole dataset before splitting, instead of fitting on train only
- Including target-correlated features that wouldn't exist at prediction time
- Using future data to predict past events
- Filling nulls using statistics computed over the full dataset

Rule: always fit transformers on training data only, then apply to val/test.

Q16 What is class imbalance, and how do you handle it?

One class dominates the data — e.g. 99% legitimate transactions, 1% fraud.

- Resampling — SMOTE (oversample minority), random undersampling
- Class weights — penalise minority-class errors more heavily
- Threshold tuning — lower the cutoff (0.5 → 0.3) to catch more positives
- Better metrics — precision, recall, F1, AUC-ROC instead of accuracy
- Collect more data for the minority class

Q17 What is the difference between bagging and boosting?

	Bagging	Boosting
How	Parallel models on bootstrapped samples	Sequential models, each fixing prior errors
Goal	Reduce variance	Reduce bias
Examples	Random Forest	XGBoost, AdaBoost, LightGBM, CatBoost
Overfitting risk	Low	Higher — tune carefully

Q18 What is a confusion matrix?

	Predicted: YES	Predicted: NO
Actual: YES	TP (hit)	FN (missed)
Actual: NO	FP (false alarm)	TN (correct reject)

- Precision = $TP / (TP + FP)$ — of everything flagged positive, how much was real?
- Recall = $TP / (TP + FN)$ — of everything actually positive, how much did we catch?
- $F1 = 2 \times P \times R / (P + R)$ — harmonic mean, balances both

Q19 When would you choose precision over recall, and vice versa?

Scenario	Prioritise	Why
Cancer screening	Recall	Missing a real case is far worse than a false alarm
Spam filter	Precision	Blocking a legitimate email is costlier than one spam slip-through
Fraud detection	Recall	Missing fraud outweighs reviewing extra alerts
Recommendations	Precision	Irrelevant items erode user trust fast

Q20 What is AUC-ROC?

- ROC curve — plots True Positive Rate vs False Positive Rate across every threshold
- AUC — one number summarising how well the model discriminates between classes
- AUC = 0.5 is random guessing, AUC = 1.0 is perfect, 0.85+ is generally strong

Use AUC-ROC when: class imbalance exists and you care about ranking ability, not one fixed threshold.

SECTION 02 • Q21 – Q40

Supervised Learning

Know the algorithm, when to use it, how it works, and its failure modes. The “how it works” answer should take 60 seconds, not 10 minutes.

Q21 How does Logistic Regression work?

It models the probability that an input belongs to a class using the sigmoid function:

$$P(y=1 \mid X) = 1 / (1 + e^{(-z)}) \quad \text{where } z = w^T X + b$$

- Outputs a probability between 0 and 1
- Trained by minimising binary cross-entropy (log loss)
- Decision boundary: predict class 1 if $P > 0.5$ (tunable)
- Assumes: a linear relationship between features and log-odds, and features that aren't highly correlated

Q22 What is the difference between Logistic and Linear Regression?

	Linear Regression	Logistic Regression
Output	Continuous value	Probability (0-1)
Activation	None (identity)	Sigmoid
Loss	MSE	Binary cross-entropy
Use	Regression	Binary classification

Q23 How does a Decision Tree work?

It splits the data recursively at each node, using the feature and threshold that best separates classes or minimises impurity.

- Gini impurity — probability of an incorrect classification
- Entropy / Information Gain — reduction in uncertainty
- MSE — used for regression trees

Pros / cons: interpretable, no scaling needed, handles mixed types — but overfits easily. Fix with max_depth, min_samples_split, or pruning.

Q24 How does Random Forest work, and why is it better than a single tree?

It builds many decision trees on bootstrapped samples, using a random subset of features at each split, then aggregates predictions (majority vote or average).

- Reduces variance through averaging (bagging)
- Feature randomness decorrelates trees, making the ensemble more diverse
- Robust to overfitting and outliers, with built-in feature importance

Key hyperparameters: n_estimators, max_depth, max_features, min_samples_leaf.

Q25 How does XGBoost work?

It builds trees sequentially — each new tree corrects the residual errors of all previous trees, using gradient descent in function space.

- Built-in L1/L2 regularisation
- Parallel tree construction and native handling of missing values
- Column subsampling, like Random Forest

Key hyperparameters: `n_estimators`, `learning_rate`, `max_depth`, `subsample`, `colsample_bytree`, `reg_alpha`, `reg_lambda`.

Q26 How does Support Vector Machine (SVM) work?

SVM finds the hyperplane that maximises the margin between classes; the closest points to the boundary are the support vectors.

- Hard margin — no misclassification allowed (linearly separable data only)
- Soft margin — allows some misclassification, controlled by `C`
- Kernel trick — projects data into higher dimensions to find a linear separator (RBF, polynomial)

Use when: high-dimensional data, text classification, small-to-medium datasets. Avoid for very large datasets or when you need calibrated probabilities.

Q27 Explain the K-Nearest Neighbours algorithm.

KNN classifies a new point by finding its `K` nearest neighbours in the training data and taking the majority class (or average, for regression).

- No training phase — it's a lazy learner; all computation happens at prediction time
- Distance metric — Euclidean by default, also Manhattan or cosine
- Scaling required — unscaled features distort distances
- Choosing `K` — small `K` means high variance, large `K` means high bias; use cross-validation

Q28 What is Naive Bayes, and what makes it “naive”?

A probabilistic classifier built on Bayes' theorem. It's “naive” because it assumes all features are independent given the class — almost never true in practice.

$$P(Y | X) = P(X | Y) \times P(Y) / P(X)$$

Works well anyway for: text classification, spam filtering, sentiment analysis — because class ranking still holds even when the independence assumption doesn't.

Q29 How do you handle multicollinearity?

Multicollinearity is when two or more features are highly correlated, making it hard to isolate each feature's individual effect.

Detection: correlation matrix, Variance Inflation Factor (VIF > 10 is a red flag).

- Remove one of the correlated features
- Use PCA to build uncorrelated components
- Ridge regression (L2) handles collinearity better than plain OLS
- Tree-based models are largely unaffected by it

Q30

What is the difference between Ridge, Lasso, and ElasticNet regression?

	Ridge (L2)	Lasso (L1)	ElasticNet
Sets weights to 0?	No	Yes	Some
Feature selection	No	Yes	Partial
Best when	All features relevant	Many irrelevant features	Mixed case

Q31

What is the precision-recall tradeoff, and how do you tune a threshold?

Lowering the classification threshold (e.g. 0.5 → 0.3) increases recall — you catch more true positives — but reduces precision, since more false positives slip through.

```
from sklearn.metrics import precision_recall_curve
precisions, recalls, thresholds = precision_recall_curve(y_true, y_proba)
# choose a threshold based on the real cost of each error type
```

Q32

How does Gradient Boosting differ from AdaBoost?

	AdaBoost	Gradient Boosting
Error correction	Reweights misclassified samples	Fits a new tree to the residual gradient
Base learner	Stumps (depth-1 trees)	Deeper trees
Speed	Fast	Slower (XGBoost/LightGBM optimise this)

Q33

What are the assumptions of Linear Regression?

- Linearity — relationship between X and y is linear
- Independence — observations don't influence each other
- Homoscedasticity — constant variance of residuals

- Normality of residuals
- No multicollinearity among features

In practice: tree-based models and neural nets skip these assumptions entirely — one reason they often win on messy real-world data.

Q34 What is R^2 , and what are its limitations?

R^2 measures how much variance in y the model explains, ranging 0–1 (can go negative for a truly bad model).

$$R^2 = 1 - \text{SS}_{\text{residual}} / \text{SS}_{\text{total}}$$

- Always increases when you add features — use Adjusted R^2 instead
- A high R^2 doesn't imply a causal relationship
- Poor indicator of performance on genuinely new data, and sensitive to outliers

Q35 When would you use each algorithm?

Algorithm	Use when
Logistic Regression	Baseline, interpretability required, linearly separable
Decision Tree	Full interpretability needed, mixed data types
Random Forest	Tabular data, robust baseline, feature importance
XGBoost / LightGBM	Best performance on tabular data, Kaggle-ready
SVM	High-dimensional data, text, small datasets
KNN	Simple baseline, recommendation systems
Naive Bayes	Text classification, fast training
Neural Network	Images, text, audio, very large datasets

Q36 What is stacking (model stacking)?

An ensemble method where predictions from several base models become input features for a meta-model, which learns how to best combine them.

```

Base models: Random Forest, XGBoost, SVM
  ↓ predictions on held-out data
Meta-model: Logistic Regression learns weights per base model
  ↓
Final prediction
  
```

Q37 How do you handle categorical variables?

Method	When to use
One-hot encoding	Low cardinality (≤ 10 categories), linear models
Label encoding	Ordinal categories (Low, Medium, High)
Target encoding	High cardinality, tree models (watch for leakage)
Binary encoding	High cardinality, more memory-efficient than one-hot
Embedding	Neural networks with high cardinality

Q38 What is the difference between hard and soft voting classifiers?

- Hard voting — takes the majority class label across all models
- Soft voting — averages predicted probabilities — usually better since it leverages confidence

Q39 How do you choose the number of trees in Random Forest?

Use cross-validation — performance typically improves with more trees, then plateaus. 100–500 trees is a common sweet spot. Watch the out-of-bag (OOB) error during training; once it stops decreasing, you have enough trees.

Q40 What is early stopping?

A regularisation technique that halts training once validation performance stops improving, avoiding wasted epochs and overfitting.

```
model = xgb.XGBClassifier(n_estimators=1000)
model.fit(X_train, y_train,
          eval_set=[(X_val, y_val)],
          early_stopping_rounds=50) # stop after 50 rounds with no gain
```

SECTION 03 • Q41 – Q55

Unsupervised Learning

Clustering, dimensionality reduction, and anomaly detection — the questions that separate people who've only used labelled data from people who've explored it.

Q41 How does K-Means clustering work?

- Randomly initialise K cluster centroids
- Assign each point to its nearest centroid
- Recompute centroids as the mean of assigned points
- Repeat until centroids stop moving

Choosing K: elbow method (inertia vs K) or Silhouette score. Limitations: assumes spherical clusters, sensitive to initialisation (use K-Means++), struggles with outliers.

Q42 What is the Elbow Method for choosing K?

Plot inertia (sum of squared distances to the nearest centroid) against K. The point where inertia stops dropping sharply — the “elbow” — suggests a good K.

Note: the elbow isn't always obvious — pair it with the Silhouette score.

Q43 What is the Silhouette Score?

It measures how well each point fits its own cluster versus neighbouring clusters, ranging from -1 to +1: +1 means well inside its cluster, 0 sits on a boundary, -1 means likely in the wrong cluster.

Q44 How does DBSCAN work, and when is it better than K-Means?

DBSCAN groups points that are densely packed — within distance epsilon of at least min_samples neighbours. Points in sparse regions are labelled outliers.

- Clusters are irregularly shaped
- You don't know K in advance
- You need to flag outliers
- Clusters have different densities (use HDBSCAN for that case)

Q45 What is PCA, and how does it work?

PCA finds the directions of maximum variance (principal components) and projects the data onto a lower-dimensional space.

- Standardise the data
- Compute the covariance matrix
- Find eigenvectors (components) and eigenvalues
- Sort by eigenvalue (explained variance) and project onto the top K

Use for: dimensionality reduction, visualisation, and removing correlated features before linear models.

Q46 What is the difference between PCA and t-SNE?

	PCA	t-SNE
Type	Linear	Non-linear
Preserves	Global structure, variance	Local structure, clusters
Interpretable?	Yes	No
Use for	Preprocessing, reduction	Visualisation only
Scales to new data?	Yes	No

Q47 What is an autoencoder?

A neural network trained to compress data into a smaller representation (encoding) and then reconstruct it (decoding); the bottleneck layer holds the compressed representation.

Uses: dimensionality reduction, anomaly detection (high reconstruction error = anomaly), denoising, generative modelling.

Q48 What is anomaly detection?

Identifying data points that are meaningfully different from the majority.

Method	How	When
Isolation Forest	Isolates anomalies with fewer splits	General purpose, fast
One-Class SVM	Boundary around normal data	Small datasets
Autoencoder	High reconstruction error	Complex or image data
Statistical	Z-score, IQR	Simple, univariate
LOF	Local density vs neighbours	Cluster-based anomalies

Q49 What is hierarchical clustering?

- Agglomerative (bottom-up) — start with each point as its own cluster, merge closest pairs, build a dendrogram
- Divisive (top-down) — start with one cluster, recursively split

Advantage over K-Means: no need to specify K upfront, and the dendrogram shows structure at every level.

Q50 What is UMAP, and how does it compare to t-SNE?

	t-SNE	UMAP
Speed	Slow	Much faster
Global structure	Poor	Better preserved
Scales to new data	No	Yes
General recommendation	—	Default choice

Q51 What is the difference between hard and soft clustering?

- Hard clustering — each point belongs to exactly one cluster (K-Means)
- Soft clustering — each point gets a probability of belonging to each cluster (Gaussian Mixture Models)

Q52 What is a Gaussian Mixture Model?

GMM assumes the data comes from a mixture of Gaussian distributions, fitted with the Expectation-Maximisation (EM) algorithm. Each point gets a soft, probabilistic assignment.

Advantage over K-Means: handles elliptical clusters and gives probabilistic assignments.

Q53 How does the EM algorithm work?

- E-step — compute the probability each point belongs to each cluster, given current parameters
- M-step — update cluster parameters (mean, covariance, weights) to maximise likelihood
- Repeat until convergence

Q54 What is matrix factorisation, and where is it used?

It decomposes a matrix into two lower-rank matrices — the classic application is recommendation systems.

User-Item rating matrix ($m \times n$) $\approx$ User matrix ($m \times k$) $\times$ Item matrix ($k \times n$)

Algorithms: SVD, NMF, ALS — used by Netflix, Spotify, and Amazon for recommendations.

Q55 What is the difference between feature selection and feature extraction?

	Feature Selection	Feature Extraction
How	Selects a subset of original features	Creates new features from original ones

	Feature Selection	Feature Extraction
Interpretability	High — original features kept	Lower — new features less intuitive
Examples	Lasso, RFE, mutual information	PCA, autoencoders, polynomial features

SECTION 04 • Q56 – Q70

Model Evaluation

The right metric, chosen for the right reason, is worth more than a fancier model.

Q56 When should you use accuracy vs F1 score?

- Accuracy — fine when classes are balanced and both error types cost the same
- F1 — needed under class imbalance, or when false positives and false negatives carry different costs

Example: with 99% legitimate transactions and 1% fraud, a model that always predicts “legit” gets 99% accuracy but 0 F1 and catches zero fraud.

Q57 What is a learning curve, and how do you read it?

Learning curves plot training and validation error against training set size.

Gap pattern	Diagnosis
Both errors high and close	Underfitting — model too simple
Training low, validation high, big gap	Overfitting — need more data or regularisation
Both low and close	Good fit
Adding data doesn't help	Reached the model's capacity limit

Q58 What is RMSE, and when would you use MAE instead?

- RMSE — penalises large errors heavily; use when big misses are disproportionately bad
- MAE — treats all errors equally, more robust to outliers

Rule: many outliers in the target → prefer MAE. Large errors especially costly → use RMSE.

Q59 What's the difference between micro, macro, and weighted averaging for multi-class metrics?

	Micro	Macro	Weighted
Treats classes	Equally by sample count	Equally by class	By frequency
Use when	Overall performance	Care equally about all classes	Imbalance, weighted by importance

Q60 How do you evaluate a clustering model?

Metric	Requires labels?	Measures
Silhouette Score	No	Cluster cohesion vs separation
Davies-Bouldin Index	No	Average cluster similarity (lower is better)
Calinski-Harabasz	No	Ratio of between/within cluster variance

Metric	Requires labels?	Measures
Adjusted Rand Index	Yes	Agreement with ground-truth labels
Inertia	No	Sum of squared distances to centroid

Q61

What is k-fold cross-validation, and how does it differ from a holdout set?

- Holdout — split once, evaluate once; results depend heavily on which split you got
- K-fold — split and evaluate K times, then average; more reliable, and uses all data for both training and validation

Q62

How do you compare two models statistically?

- Run K-fold CV on both models
- Compare the mean and standard deviation of the scores
- Run a paired t-test or Wilcoxon test on the fold-level scores
- Treat a difference as meaningful only if it exceeds the variation across folds

Q63

What is calibration, and why does it matter?

A model is well-calibrated if its predicted probabilities match real-world frequencies — “80% confidence” should turn out correct 80% of the time.

- Tools: reliability diagram (calibration curve), Brier score
- Fix: Platt scaling, isotonic regression

Why it matters: if you feed probabilities into pricing or prioritisation decisions, a miscalibrated model quietly produces the wrong expected values.

Q64

What is NDCG, and when is it used?

Normalised Discounted Cumulative Gain is a ranking metric that rewards relevant items appearing higher in a list. It's the standard for search engines and recommendation systems.

Q65

What is the difference between offline and online evaluation?

	Offline	Online
Method	Historical data, backtest	A/B test in production
Speed	Fast	Slow — needs live traffic
Reliability	May not reflect real behaviour	Ground-truth user response

Offline		Online
Risk	None	Exposes real users to a potentially worse experience

Q66 How do you detect and handle outliers before training?

Method	Approach
IQR	Flag values below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$
Z-score	Flag values beyond 3 standard deviations
Isolation Forest	Unsupervised anomaly detection
Domain rules	e.g. age > 120 is always wrong, statistics aside

What to do: investigate first, remove genuine data-entry errors, cap (winsorise) real extreme values, and log-transform to reduce skew.

Q67 What is the holdout method, and what are its limitations?

It splits data once into train/test. Limitations: high variance depending on which split you got, and it wastes data since the test set is never used for training. Cross-validation gives a more robust estimate.

Q68 What is MAPE, and what are its problems?

Mean Absolute Percentage Error is the mean of $|\text{actual} - \text{predicted}| / |\text{actual}|$.

- Undefined when the actual value is 0
- Asymmetric — punishes under-predictions harder than over-predictions
- Sensitive to small actuals (an error of 1 on an actual of 2 is a 50% error)

Alternatives: sMAPE, MASE (Mean Absolute Scaled Error).

Q69 How do you choose the right evaluation metric for a business problem?

- Understand the error costs — what's worse, a false positive or false negative? By how much?
- Understand the output type — probability, rank, or binary label?
- Understand class balance — balanced → accuracy is fine; imbalanced → F1, AUC
- Link it to the business KPI — a model that optimises log loss but doesn't move revenue is the wrong model

Q70

What is the difference between MSE and MAE, and when would you use each?

	MSE	MAE
Outlier sensitivity	High — squaring amplifies	Low — linear
Differentiable at 0	Yes	No
When to use	Large errors are especially bad	Robust to outliers, errors equally weighted

SECTION 05 • Q71 – Q82

Feature Engineering

The skill that actually separates good models from bad ones.

Why this section matters: a mediocre model with great features beats a great model with mediocre features. Every single time.

Q71 What is feature engineering, and why does it matter?

Feature engineering creates, transforms, or selects input variables to improve model performance — it translates domain knowledge into a representation the model can learn from.

- Date → day of week, is_weekend, days_since_last_purchase
- Price → $\log(\text{price})$ to reduce skew
- Lat/Long → distance to the nearest store

Q72 How do you handle high-cardinality categorical features?

- Target encoding — replace the category with the mean target (risk: leakage, use K-fold encoding)
- Frequency encoding — replace with the category's frequency in training data
- Embeddings — learn dense representations for neural networks
- Hashing — map to a fixed-size space, fast but loses interpretability
- Group rare categories into an “Other” bucket

Q73 What is target encoding, and what are the risks?

Target encoding replaces a category with the mean target value for that category.

Risk: leakage — if the mean is computed over rows including the test row, the model effectively sees the target during training.

Fix: K-fold target encoding — compute the encoding using other folds only.

Q74 When would you apply a log transformation to a feature?

- The feature is right-skewed with a long tail of large values
- The feature has multiplicative relationships (e.g. doubling house size roughly doubles price)
- The target has high positive skew, or you need to stabilise variance

```
df['log_revenue'] = np.log1p(df['revenue']) # log1p safely handles 0s
```

Q75 What are interaction features?

New features built by combining two or more existing features, capturing relationships a linear model couldn't see on its own.

```
df['rooms_per_floor'] = df['rooms'] / df['floors']
df['revenue_per_customer'] = df['revenue'] / df['customer_count']
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X) # generates all pairwise products
```

Q76 How do you handle missing values during feature engineering?

First: create a missing indicator — `df['age_missing'] = df['age'].isna().astype(int)`

- Numeric — fill with median (robust) or use model-based imputation (KNN Imputer)
- Categorical — fill with the mode or an “Unknown” category
- Time series — forward-fill, backward-fill, or interpolate

Never: fill the test set using test-set statistics — always fit on train, transform both.

Q77 What is feature importance, and how do you calculate it?

Method	Model	Interpretation
Gini importance	Tree models	Average impurity reduction across splits
Permutation importance	Any	Performance drop when a feature is shuffled
SHAP values	Any	Each feature's contribution to each prediction
Coefficient magnitude	Linear models	Weight of each feature, after scaling

SHAP is the gold standard; Gini importance can mislead for high-cardinality features.

Q78 What's the difference between filter, wrapper, and embedded feature selection methods?

	Filter	Wrapper	Embedded
How	Statistical test on features	Evaluate subsets with a model	Selection during model training
Examples	Correlation, chi-squared, mutual info	RFE, forward selection	Lasso, decision tree
Speed	Fast	Slow	Medium

Q79 What is PCA used for in feature engineering specifically?

- Creating uncorrelated features from correlated ones, before linear models
- Reducing dimensionality to sidestep the curse of dimensionality
- Removing noise via low-variance components

- Visualising high-dimensional data in 2D/3D

Not useful for: tree-based models (which handle correlated features fine), or whenever interpretability is required.

Q80 How do you create time-based features from a timestamp?

```
df['hour'] = df['ts'].dt.hour
df['day_of_week'] = df['ts'].dt.dayofweek # 0=Mon, 6=Sun
df['is_weekend'] = df['day_of_week'].isin([5,6]).astype(int)
df['days_since'] = (df['ts'].max() - df['ts']).dt.days
df['lag_7d'] = df['value'].shift(7) # 7-day lag
df['roll_7d_avg'] = df['value'].rolling(7).mean()
```

Q81 What is oversampling and undersampling?

- Undersampling — randomly remove majority-class samples; fast but loses data
- Oversampling — randomly duplicate minority samples; easy but creates exact copies
- SMOTE — creates synthetic minority samples by interpolating between existing ones; generally better than random oversampling

```
from imblearn.over_sampling import SMOTE
X_res, y_res = SMOTE().fit_resample(X_train, y_train)
```

Q82 What is the difference between one-hot encoding and ordinal encoding?

- One-hot encoding — each category becomes a binary column, no order implied — for nominal categories
- Ordinal encoding — categories mapped to integers, order matters — for genuinely ordered categories

Warning: never ordinal-encode a nominal category — it implies a false ordering that misleads linear models.



SECTION 06 • Q83 – Q95

Deep Learning & NLP

Neural networks, transformers, and the practical deep-learning questions that come up in almost every applied-AI interview now.

Q83 How does backpropagation work?

Backprop computes the gradient of the loss with respect to every weight using the chain rule, flowing backwards from the output layer to the input layer. Gradient descent then uses those gradients to update the weights.

Forward pass: input $\rightarrow$ prediction $\rightarrow$ loss
 Backward pass: loss $\rightarrow \partial \text{loss} / \partial \text{weights}$ (chain rule) $\rightarrow$ weight update

Q84 What is dropout, and how does it prevent overfitting?

During training, a random fraction p of neurons is zeroed out on each forward pass, forcing the network to learn redundant representations — no single neuron can be relied on.

- At test time all neurons are active, with weights scaled by $(1 - p)$
- Typical rates: $p = 0.2-0.5$
- Effectively trains an ensemble of exponentially many sub-networks

Q85 What are activation functions, and why are they needed?

They introduce non-linearity, without which a deep network would collapse into one linear function.

Function	Formula	Use
ReLU	$\max(0, x)$	Hidden layers (default)
Leaky ReLU	$\max(0.01x, x)$	Fixes the dying ReLU problem
Sigmoid	$1/(1+e^{-x})$	Binary output layer
Softmax	$e^x / \sum e^x$	Multi-class output layer
Tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$	RNNs, range -1 to 1

Q86 What is batch normalisation?

It normalises layer inputs to zero mean and unit variance, then rescales them with learned parameters γ and β , applied between layers during training.

Benefits: faster training, higher learning rates tolerated, a mild regularisation effect, and less sensitivity to weight initialisation.

Q87 What is a convolutional neural network (CNN), and when do you use it?

CNNs use convolutional layers to detect local patterns — edges, textures, shapes — by sliding a learned filter across the input. Pooling layers downsample, and fully connected layers classify.

Use for: images, video, spatial data — anywhere local patterns matter.

Q88 What is an RNN, and what problem does LSTM solve?

- RNN — processes sequences with a hidden state carried across time steps, but suffers from the vanishing gradient problem and can't remember long-range dependencies
- LSTM — adds gates (forget, input, output) to control what information to keep or discard over many steps, solving that vanishing gradient issue

Q89 What is the Transformer architecture?

Transformers process an entire sequence in parallel using self-attention, letting every token attend to every other token.

- Self-attention — each token computes relevance to every other token
- Multi-head attention — multiple heads capture different relationships
- Positional encoding — injects order, since attention alone has none
- Feed-forward layers — transform each token's representation independently

Q90 What is the attention mechanism?

Attention lets the model weight the importance of different parts of the input when producing each output element.

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

- Q (Query) — what we're looking for
- K (Key) — what each token represents
- V (Value) — what we return for each token

Q91 What is transfer learning?

Take a model trained on a large dataset (ImageNet, web text), then fine-tune it on a smaller, task-specific dataset. The pre-trained model already knows general representations; fine-tuning only adapts the later layers.

Why it works: low-level features — edges and textures for images, grammar and semantics for text — are reusable across many tasks.

Q92 What is the difference between BERT and GPT?

	BERT	GPT
Architecture	Encoder only	Decoder only

	BERT	GPT
Attention direction	Bidirectional	Left-to-right (causal)
Pre-training task	Masked Language Modelling	Next token prediction
Best for	Understanding (classification, NER, QA)	Generation (text, code)

Q93

What are word embeddings, and why are they better than one-hot encoding?

Word embeddings represent words as dense vectors in a continuous space where semantically similar words sit close together.

- One-hot — 50,000-dimensional, sparse, captures no relationships
- Word2Vec / GloVe — ~300-dimensional dense vectors; “king – man + woman $\approx$ queen”
- Contextual (BERT) — the same word gets a different embedding depending on context

Q94

What is the vanishing gradient problem?

In deep networks, gradients shrink exponentially as they propagate backwards through many layers, so early layers receive almost no signal and barely update.

Solutions: ReLU activations, batch normalisation, residual connections (ResNet), LSTM gates, gradient clipping.

Q95

What is a GAN?

A Generative Adversarial Network pits two networks against each other:

- Generator — tries to produce realistic fake data
- Discriminator — tries to tell real from fake

They compete until the generator's output is indistinguishable from real data — used for image synthesis, data augmentation, and style transfer.

SECTION 07 • Q96 – Q110

ML System Design

End-to-end thinking — the senior-level differentiator.

This is the senior-level differentiator: anyone can train a model. Very few can design a system that stays reliable, fair, and improving in production.

Q96 How would you approach building an ML model from scratch?

- Understand the business problem — what decision does the model enable, and what's the success metric?
- Define the ML task — classification, regression, or ranking?
- Understand the data — EDA, quality checks, availability at prediction time
- Establish a baseline — rule-based, heuristic, or simple average
- Feature engineering — turn domain knowledge into model inputs
- Model selection and training — start simple, iterate
- Evaluation — offline metrics tied to the business goal
- Deploy — API, batch, or embedded
- Monitor — data drift, model drift, business KPIs

Q97 What is model drift, and how do you detect it?

- Data drift — the distribution of input features changes over time
- Concept drift — the relationship between features and target changes
- Label drift — the distribution of the target itself changes

Detection: monitor feature statistics (KS test, PSI), compare prediction distributions, track business KPIs, and set alerts on metric drops.

Q98 How would you design a recommendation system?

Approach	How	When
Collaborative filtering	Find similar users/items	Dense interaction data
Content-based	Item features + user preferences	Cold start, rich metadata
Two-tower model	Separate user and item encoders	Large-scale, real-time
Matrix factorisation	Decompose the user-item matrix	General purpose
Hybrid	Combine multiple signals	Production systems

Key problems to solve: cold start (new users/items), scalability, and feedback loops.

Q99 What is A/B testing in the context of ML?

- Split users randomly into control (Model A) and treatment (Model B)
- Run long enough to reach statistical significance

- Measure the business metric, not just the ML metric
- Watch for novelty effects, and guard against the multiple-comparisons problem

Q10
0

How do you handle the cold start problem?

- New user — popularity-based recommendations, onboarding questions, demographic features
- New item — content-based features, item metadata, editorial curation
- Hybrid — lean on content signals immediately, blend in collaborative filtering as data builds up

Q10
1

How would you make an ML model explainable?

Method	Works for	Output
SHAP	Any model	Per-feature contribution per prediction
LIME	Any model	Local linear approximation
Feature importance	Tree models	Global feature ranking
Attention weights	Transformers	Which tokens the model attended to
Partial dependence plots	Any	Marginal effect of one feature
Simpler model	—	Use logistic regression or a decision tree directly

Q10
2

What is MLOps?

The practice of deploying, monitoring, and maintaining ML models in production reliably.

- Data versioning (DVC)
- Experiment tracking (MLflow, W&B)
- Model registry — version control for model artifacts
- CI/CD for ML — automated retrain → test → deploy
- Monitoring — data drift, performance, latency
- Feature store — centralised, versioned feature computation

Q10
3

How do you serve a model in production?

Method	Use when	Latency
REST API	Real-time, interactive	Low ms
Batch scoring	Offline, periodic	Hours/days
Streaming	Near-real-time on event streams	Low seconds
Edge inference	On-device, no internet	Sub-ms

Q10
4**What is a feature store, and why is it important?**

A centralised system for storing, computing, and serving ML features.

- Same features used in training and serving — prevents training-serving skew
- Features computed once and reused across multiple models
- Supports backfilling historical features
- Ensures point-in-time correct feature retrieval

Q10
5**How do you detect and mitigate bias in ML models?**

Detection: slice performance by demographic group, inspect the confusion matrix per subgroup, run a disparate impact analysis.

- Pre-processing — balance training data, remove proxy features
- In-training — fairness constraints, adversarial debiasing
- Post-processing — equalise odds by adjusting thresholds per group

Q10
6**What is online learning vs batch learning?**

	Batch Learning	Online Learning
Updates	On the full dataset	On each new sample or mini-batch
When to use	Stable data, periodic retraining is fine	Rapidly changing data, needs real-time adaptation
Examples	Most offline training	Ad click prediction, stock trading
Risk	Staleness	Sensitivity to bad data

Q10
7**What is the difference between model accuracy and business impact?**

A model with 95% accuracy that doesn't change user behaviour has zero business impact. A model with 80% accuracy that lifts conversion 15% is a genuine success.

- Define the decision the model actually enables
- Estimate the impact per unit of decision quality
- A/B test to confirm actual behaviour change
- Track the business KPI, not just the ML metric

Q10
8**How do you handle very large datasets that don't fit in memory?**

- Chunked processing — read and process in batches
- Dask or Spark — distributed computation
- Feature hashing — map features to fixed-size arrays without materialising every category
- Sampling — use a representative subset for initial modelling

- Cloud ML — Vertex AI, SageMaker for managed large-scale training
- Incremental learning — `partial_fit()` for online algorithms in sklearn

Q10
9

What is shadow deployment?

The new model runs in parallel with the production model, but its outputs are never shown to users. Real traffic is used to compare predictions silently before committing to the new model — eliminating the risk of exposing users to something untested.

Q11
0

How do you decide when to retrain a model?

- Scheduled — every N days regardless; simple and predictable
- Performance-triggered — retrain when a validation metric drops below threshold
- Drift-triggered — retrain when data drift exceeds a threshold (PSI, KS test)
- Event-triggered — major business change, new data source, product change

Best practice: automate the trigger, validate before promoting a new model, and always keep a rollback path ready.

YOU'RE READY WHEN...

The Final Checklist

Fundamentals

Explain bias-variance tradeoff, cross-validation, gradient descent, regularisation

Algorithms

Explain how Random Forest, XGBoost, and Logistic Regression work in 60 seconds each

Evaluation

Name the right metric for any given business problem, and justify it

Features

Explain why feature engineering matters more than algorithm choice

Deep Learning

Explain backprop, dropout, attention, and transfer learning clearly

System Design

Walk through an end-to-end ML system, including drift monitoring

Business

Translate a business problem into an ML task with the right success metric

One last thing: confidence in an interview comes from having said the answer out loud before, not from having read it silently one more time. Explain each of these 110 answers to a rubber duck, a friend, or a mirror — today.